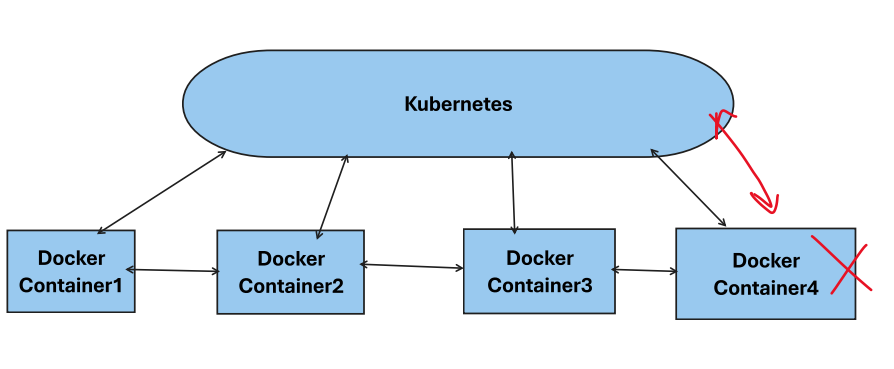
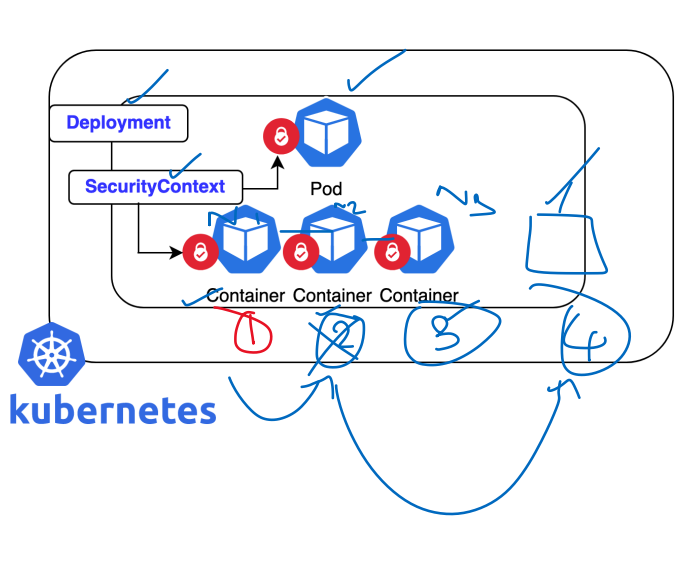
Day 4: Introduction to Kubernetes & GKE
Kubernetes architecture (Pods, Deployments, Services)
Introduction to GKE (Standard vs Autopilot)
Resource limits, requests, health probes
Namespaces, ConfigMaps, Secrets
Hands-On: Deploy container to GKE with manifest YAMLs

Introduction to Kubernetes & GKE:



Why Kubernetes:

Docker helps you run containers, but Kubernetes helps you manage containers at scale(More numbers).



* Real-world challenges:
- 1000 users hit your app → one container crashes
- You need multiple instances
- One server goes down → app is unavailable
- You need auto-scaling
- You want zero downtime deployments

* Docker alone cannot handle all this easily

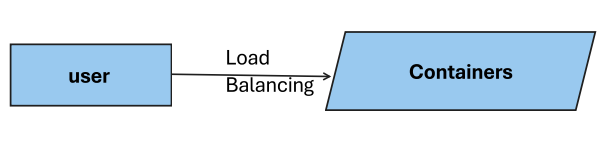
** -> Kubernetes (K8s) is a container orchestration platform:
- Deployment
- Scaling
- Networking
- Self-healing

* Key problems Kubernetes solves
-> (1) Auto Scaling:

More traffic → more containers
Less traffic → fewer containers

-> (2) Self-Healing:
If your container crashes:
- Kubernetes automatically restarts it

-> (3) Load Balancing



User → Load Balancer → Multiple Containers

-> (4) Zero Downtime Deployment
Update your app without stopping it:

Old version → New version (rolling update)

-> (5) High Availability
Runs containers across multiple machines:

If one server fails → others take over

Simple comparison

Feature	Docker	Kubernetes
Run container	✅	✅
Manage many containers	❌	✅
Auto scaling	❌	✅
Self-healing	❌	✅
Load balancing	❌	✅

What we were doing till Now:

We wrote Application Code -> Docker → Build Image → Run Container

Kubernetes → Manage at scale (1000 containers)

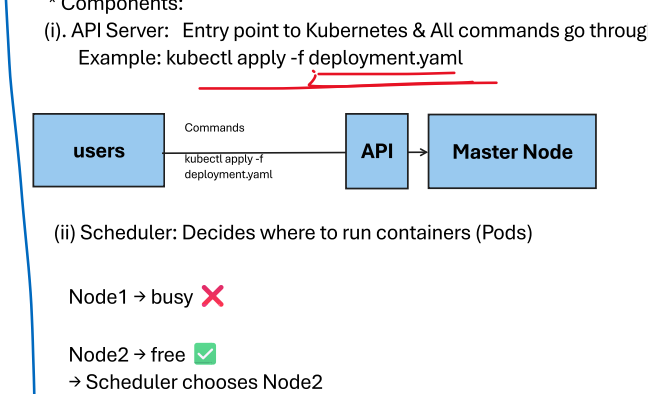
*** Kubernetes Architecture ***

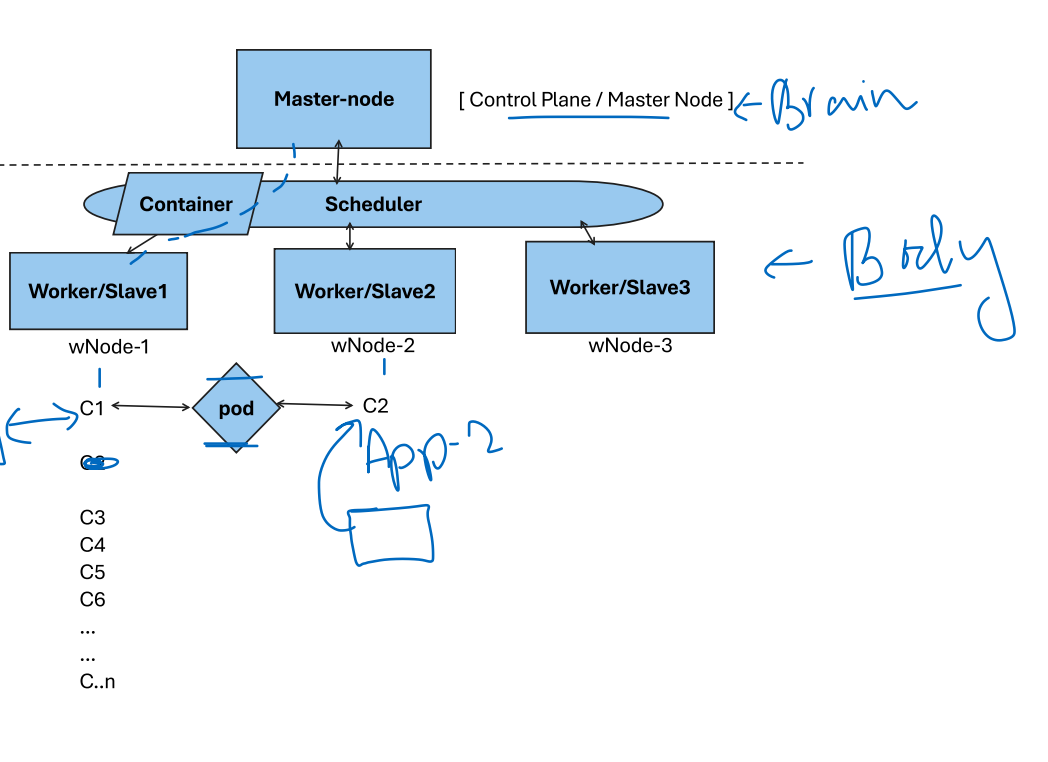
Kubernetes is made of two main parts:

1️⃣ Control Plane (Master Node)
2️⃣ Worker Nodes

1. Control Plane / Master Node (Brain of Kubernetes)

This manages the entire cluster.
* Components:
(i). API Server: Entry point to Kubernetes & All commands go through here (kubectl, UI, APIs)
Example: kubectl apply -f deployment.yaml





(ii) Scheduler: Decides where to run containers (Pods)

Node1 → busy ❌

Node2 → free ✅
→ Scheduler chooses Node2

(iii) Controller Manager: Maintains desired state

Example: You want 3 Pods - Only 2 running
→ It creates 1 more automatically

Pod: is a wrapper around one or more containers that share the same network, storage, and lifecycle.

2. Worker Nodes (Execution Layer):

These actually run your application containers.

Components:

1. Kubelet:
- Agent running on each node
- Talks to Control Plane
- Ensures containers are running (Sends - Regular Heart Beats)

2. Container Runtime:
Runs containers (Docker / containerd)

Docker → runs monolithic-python-app

3. Kube Proxy:
- Handles networking
- Routes traffic to correct Pods

** Full Flow **

* We have built the application: monolithic-python-app:2.0

-> In Kubernetes:

Step1: You create a Deployment YAML

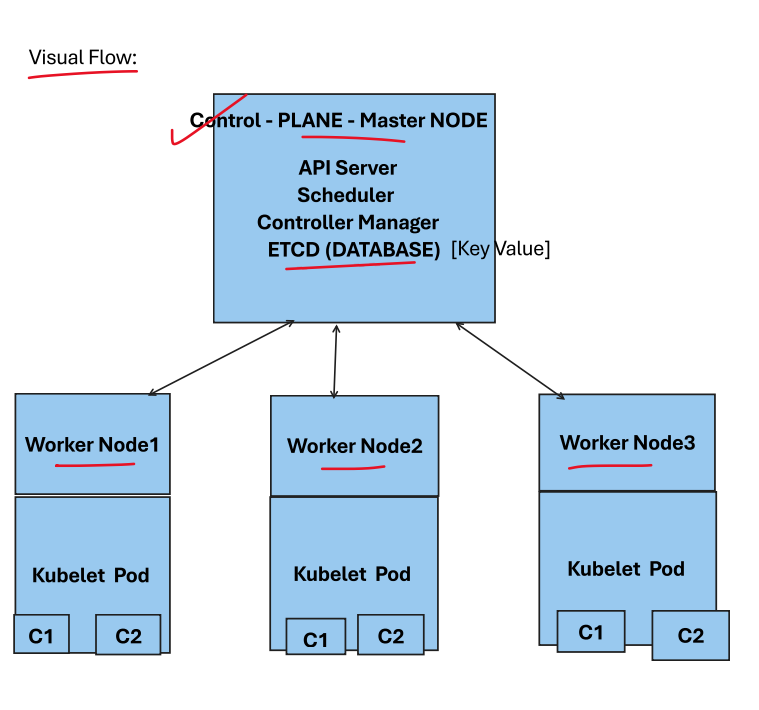
Step2: Send it via YAML Script : -> control plane

kubectl apply -f deployment.yaml (Inside the YAML file script we are mentioning details of : monolithic-python-app:2.0)

Step 3:

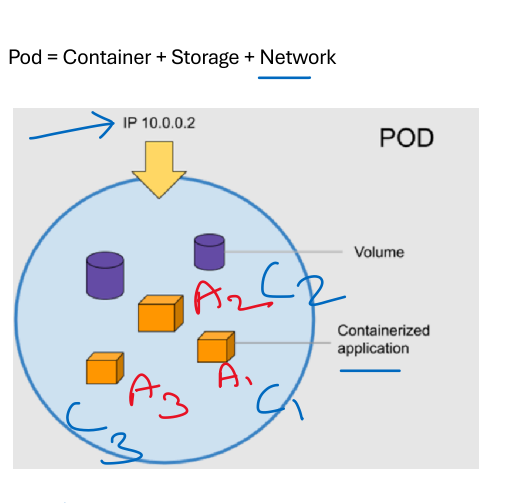
User → API Server → Scheduler → Worker Node
↓
Kubelet
↓
Container runs
C1
Application

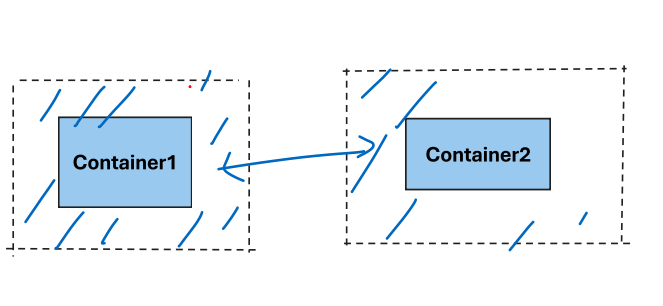
Visual Flow:



** Key Kubernetes Objects:
Pod: Smallest unit in Kubernetes.

Pod = Container + Storage + Network

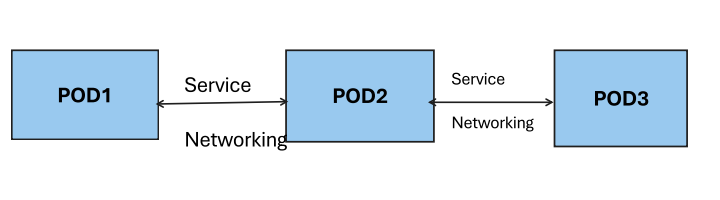




* Deployment * :
Manages Pods automatically.

Features: (1) Scaling (2)Rolling updates (2) Self-healing

** Service :
Exposes Pods to network.
Types: ClusterIP , NodePort , LoadBalancer (used in GKE)



** Introduction to GKE (Google Kubernetes Engine) **

Google Kubernetes Engine is a managed Kubernetes service where Google handles infrastructure so you can focus on applications.

Why GKE? :

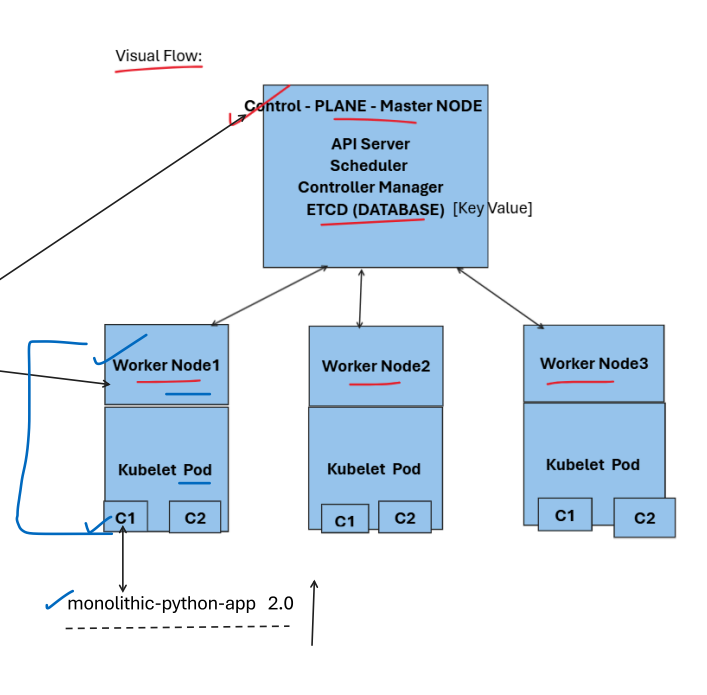
Instead of manually setting up Kubernetes cluster infra:
- Install control plane
- Manage nodes
- Handle upgrades

** GKE does all of this for you ~

* GKE Modes *
(1) Standard Mode (More Control)
* You/User manage:
- Worker Nodes (VMs)
- Node pools
- Scaling (manual / auto)
- Upgrades (partially)

Google manages:
- Control Plane (Master)

Visual Flow:



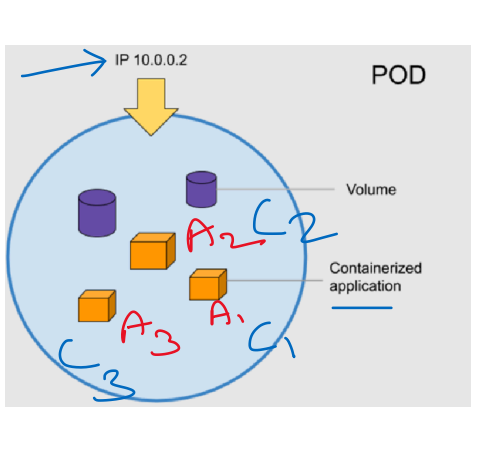
* When to use:
- Production systems needing full control
- Custom machine types (CPU, GPU)
- Advanced networking/security setups

(2) Autopilot Mode (Fully Managed):
*Google manages:
- Nodes ✅ (Master & Worker Nodes)
- Scaling ✅
- Security ✅
- Scheduling optimization ✅

*You/ User manage:
- Only your application (Pods, Deployments) ✅

* When to use:
- Beginners / quick deployments
- No infra management needed
- Cost optimization (pay per pod)

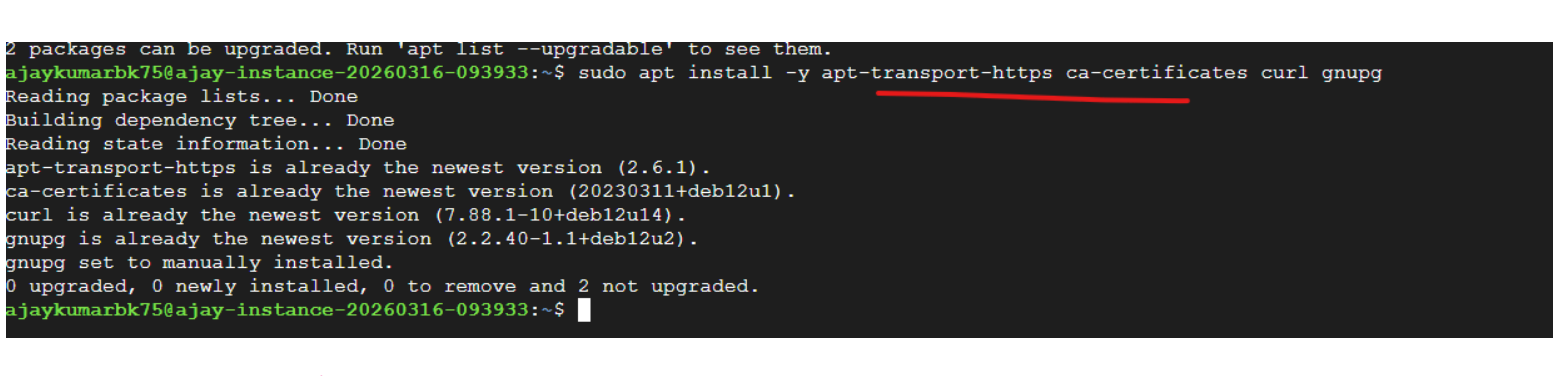
GKE automatically:
- Creates nodes
- Scales them
- Runs your containers



* Standard vs Autopilot:

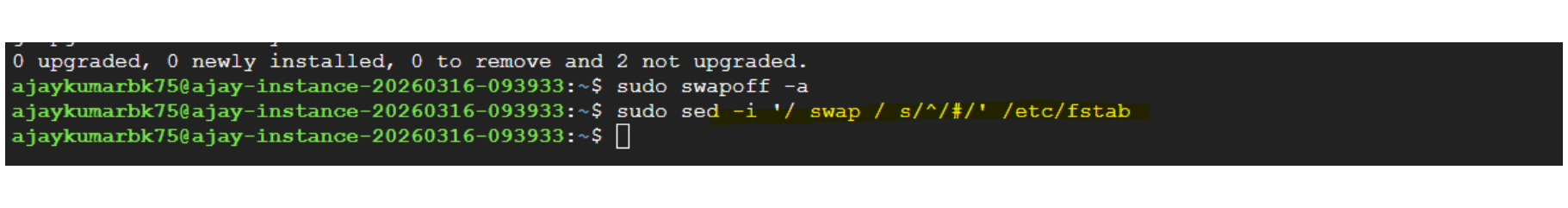
Feature	Standard Mode	Autopilot Mode
Node management	You	Google
Scaling	Manual/Auto	Fully automatic
Control	High	Low
Complexity	High	Low
Cost model	VM-based	Pod-based
Best for	Production control	Simplicity

*Kubernetes installation in VM *
Step 1: Install Required Base Packages



```
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo apt install -y apt-transport-https ca-certificates curl gnupg
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apt-transport-https is already the newest version (2.6.1).
ca-certificates is already the newest version (20230311+deb12u1).
curl is already the newest version (7.88.1-10+deb12u14).
gnupg is already the newest version (2.2.40-1.1+deb12u2).
gnupg set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

Step 2: Disable Swap (CRITICAL ❗)



```
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo swapoff -a
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo sed -i '/ swap / s/^/#/' /etc/fstab
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

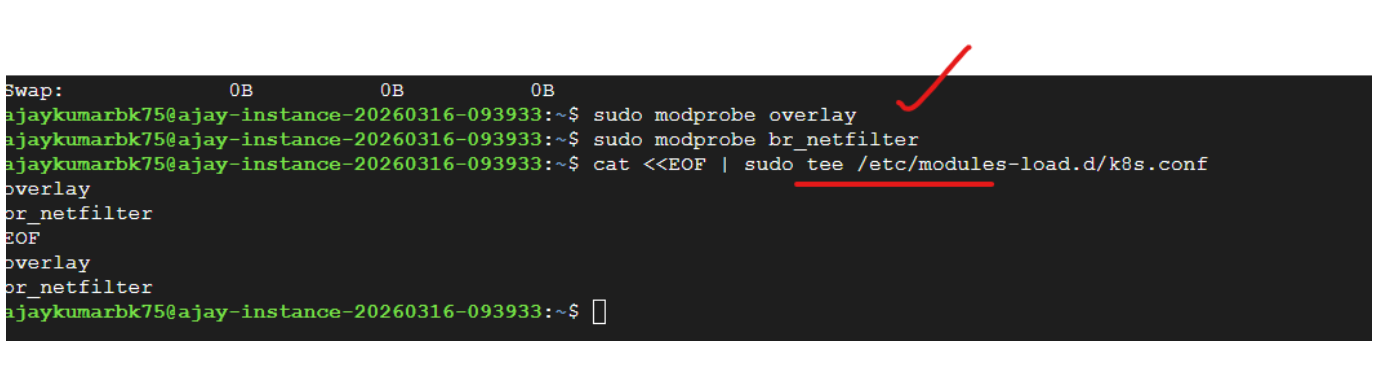
Step 3: Enable Kernel Modules

sudo modprobe overlay
sudo modprobe br_netfilter

lets use -> Persist - checks System reads it during boot

(Load these modules automatically on startup)

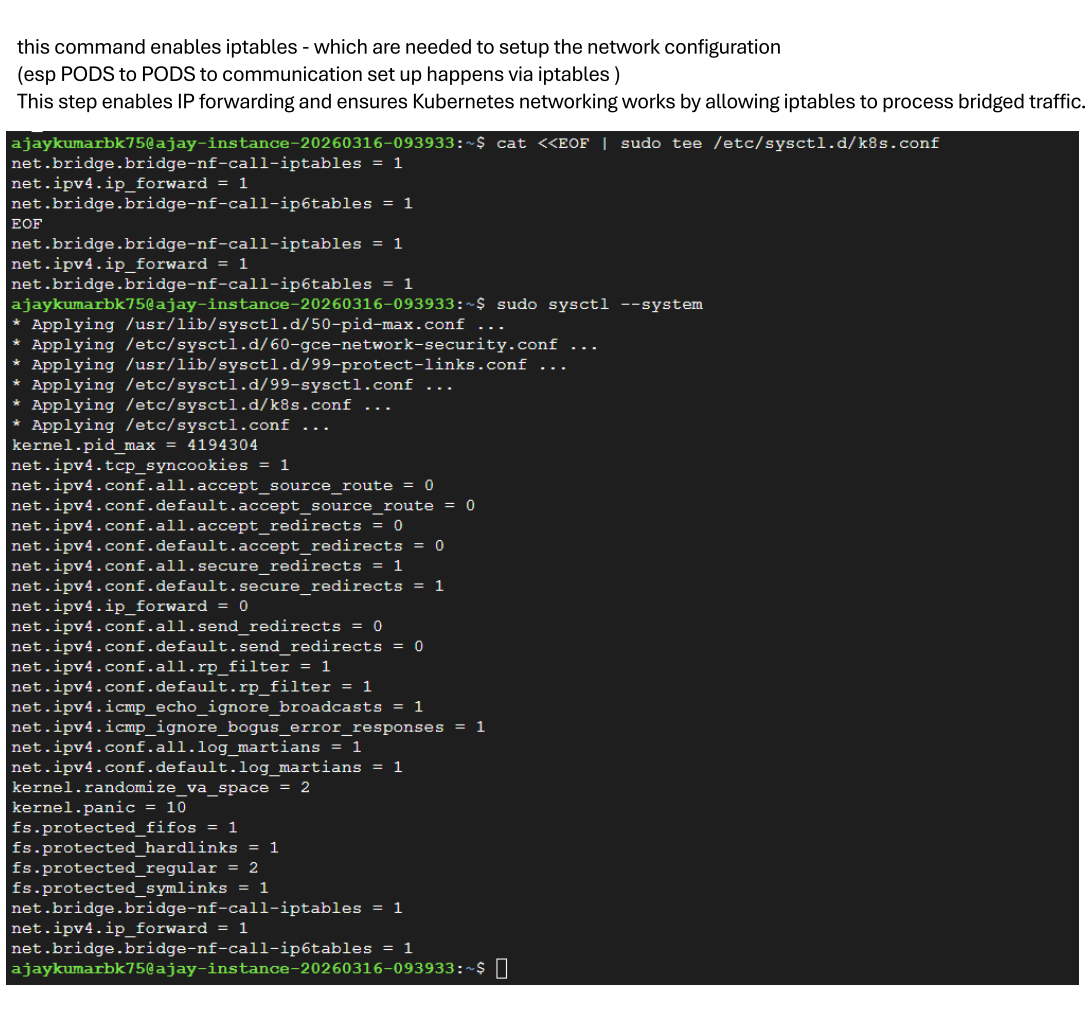
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF



```
Swap:             0B          0B          0B
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo modprobe overlay
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo modprobe br_netfilter
ajaykumarbk75@ajay-instance-20260316-093933:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
overlay
br_netfilter
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

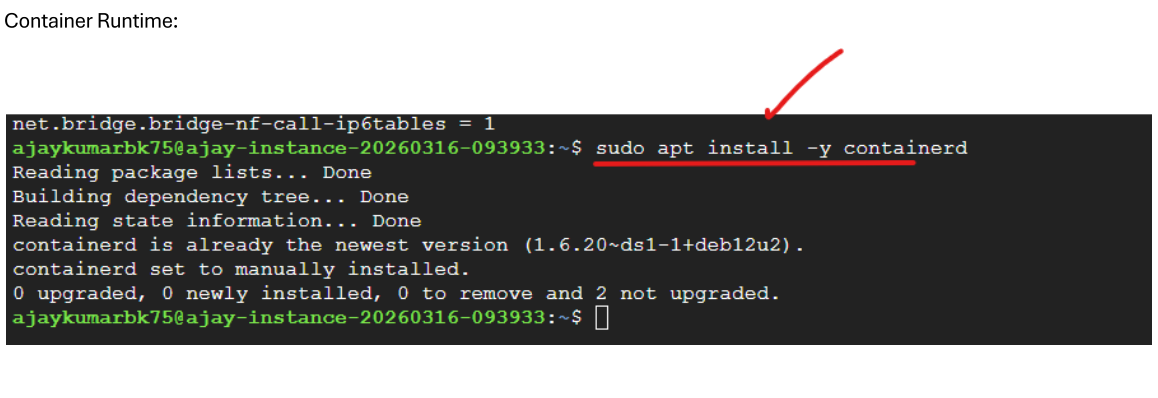
** Step 4: Configure Networking (VERY IMPORTANT)

this command enables iptables - which are needed to setup the network configuration
(esp PODS to PODS to communication set up happens via iptables)
This step enables IP forwarding and ensures Kubernetes networking works by allowing iptables to process bridged traffic.



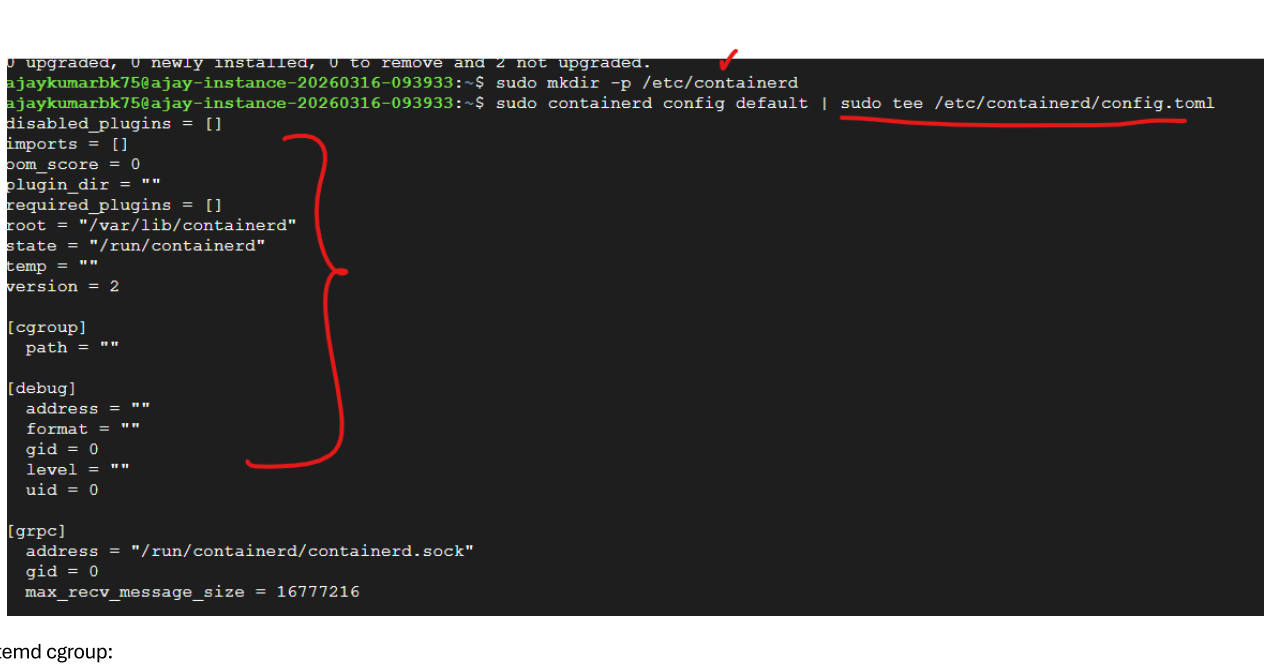
```
ajaykumarbk75@ajay-instance-20260316-093933:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /etc/sysctl.d/60-gce-network-security.conf ...
* Applying /usr/lib/sysctl.d/99-protect-links.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/k8s.conf ...
* Applying /etc/sysctl.conf ...
kernel.pid_max = 4194304
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.default.accept_redirects = 0
net.ipv4.conf.all.secure_redirects = 1
net.ipv4.conf.default.secure_redirects = 1
net.ipv4.ip_forward = 0
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.default.send_redirects = 0
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.default.rp_filter = 1
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_ignore_bogus_error_responses = 1
net.ipv4.conf.all.log_martians = 1
net.ipv4.conf.default.log_martians = 1
kernel.randomize_va_space = 2
kernel.panic = 10
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

Step 5: Install Container Runtime:



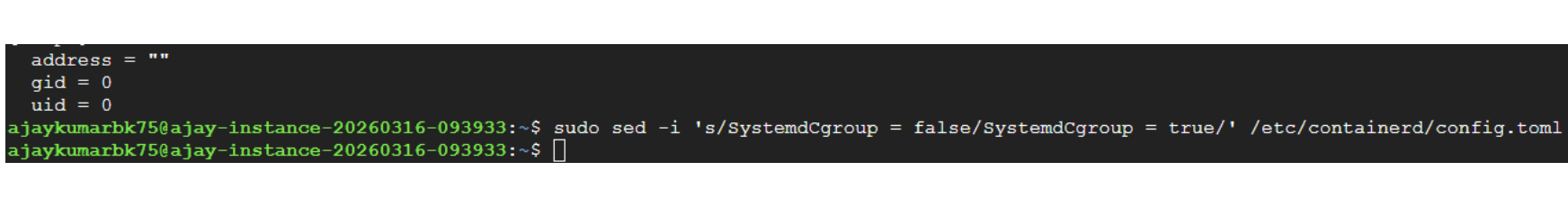
```
net.bridge.bridge-nf-call-ip6tables = 1
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo apt install -y containerd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
containerd is already the newest version (1.6.20~ds1-1+deb12u2).
containerd set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

Configure it:



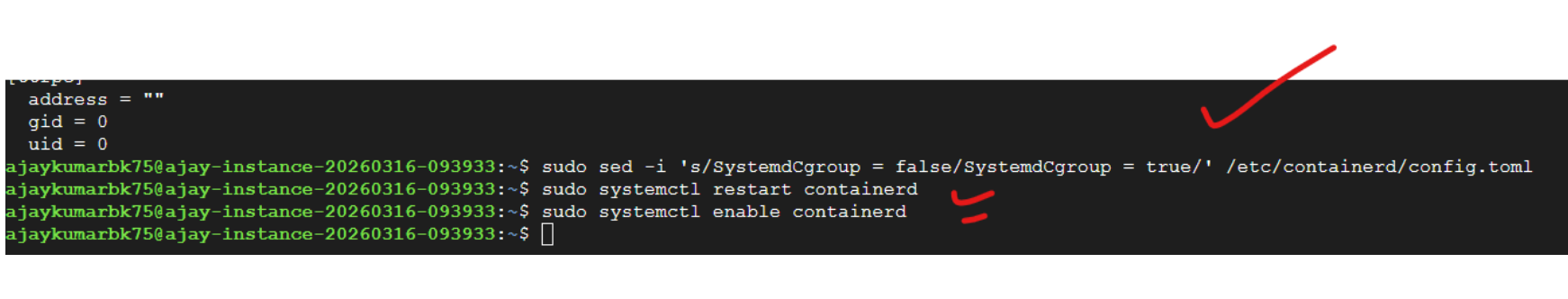
```
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo mkdir -p /etc/containerd
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo containerd config default | sudo tee /etc/containerd/config.toml
disabled_plugins = []
imports = []
oom_score = 0
plugin_dir = ""
required_plugins = []
root = "/var/lib/containerd"
state = "/run/containerd"
temp = ""
version = 2

[cgroup]
  path = ""

[debug]
  address = ""
  format = ""
  gid = 0
  level = ""
  uid = 0

[grpc]
  address = "/run/containerd/containerd.sock"
  gid = 0
  max_recv_message_size = 16777216
```

- Enable systemd cgroup:



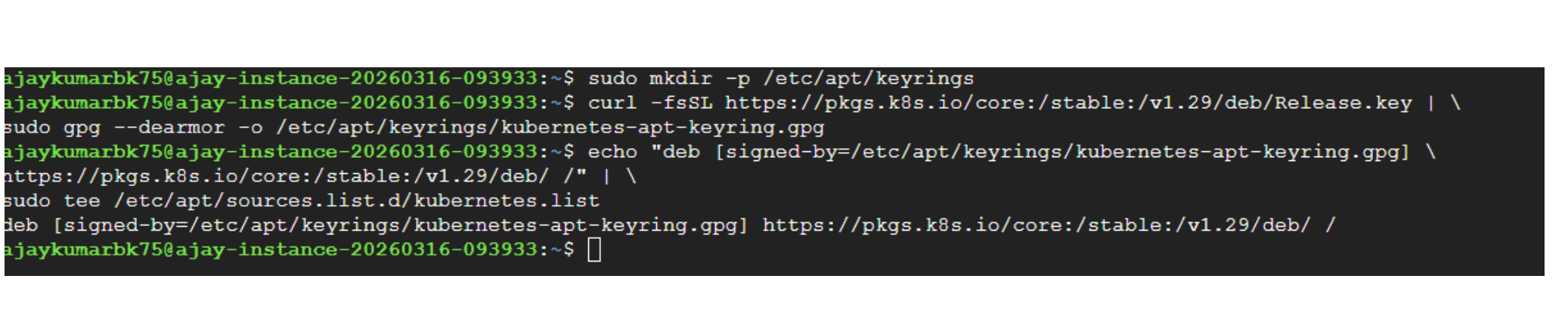
```
  address = ""
  gid = 0
  uid = 0
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

- restart



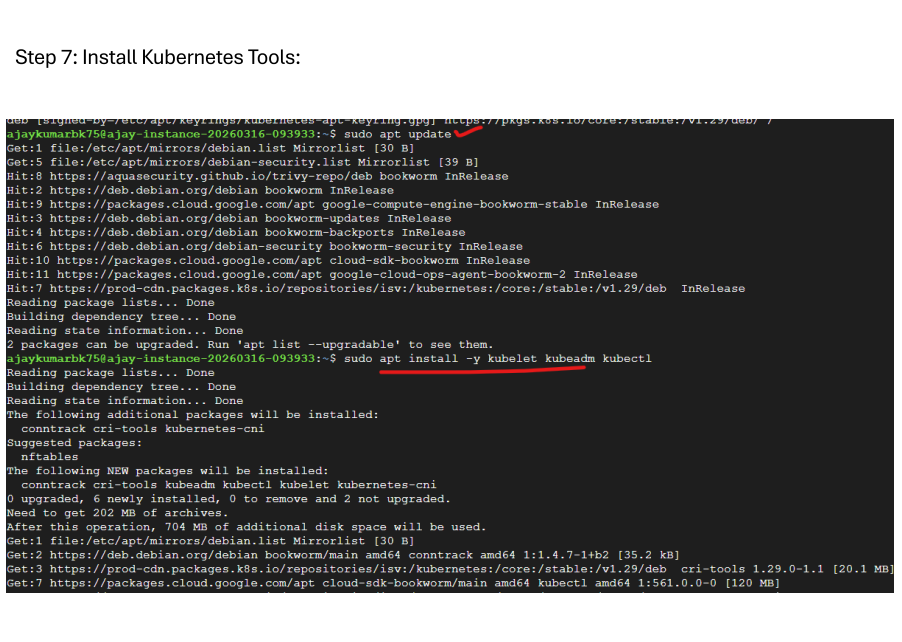
```
  address = ""
  gid = 0
  uid = 0
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo systemctl restart containerd
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo systemctl enable containerd
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

** Step 6: Add Kubernetes Repository



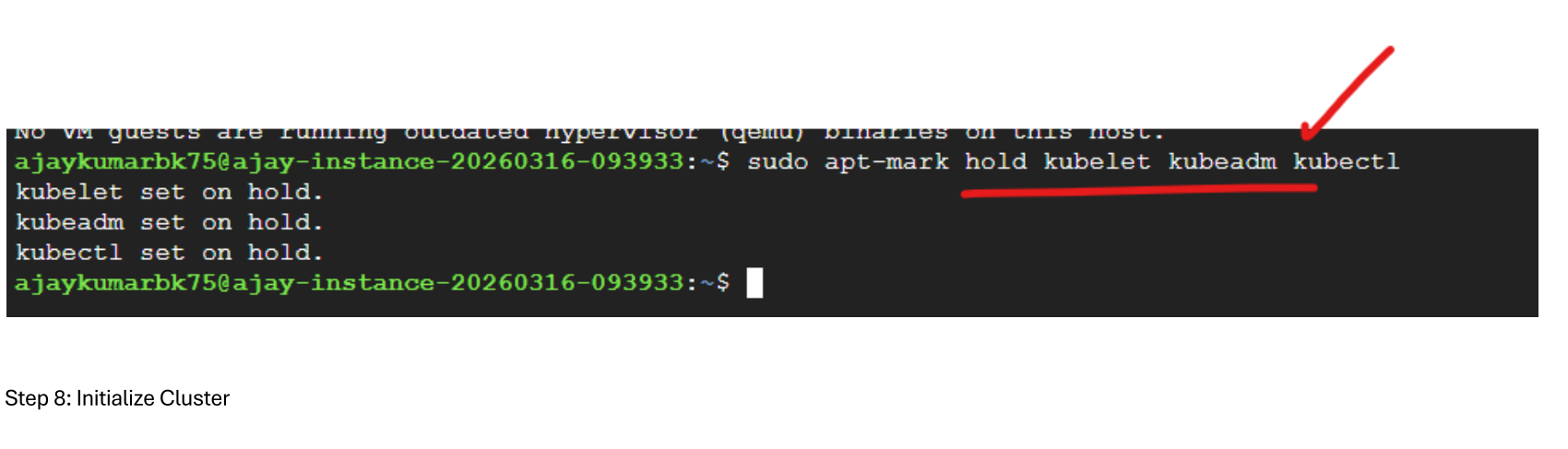
```
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo mkdir -p /etc/apt/keyrings
ajaykumarbk75@ajay-instance-20260316-093933:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
ajaykumarbk75@ajay-instance-20260316-093933:~$ echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /" | \
sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

Step 7: Install Kubernetes Tools:



```
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo apt update
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:5 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Hit:8 https://aquasecurity.github.io/trivy-repo/deb bookworm InRelease
Hit:2 https://deb.debian.org/debian bookworm InRelease
Hit:9 https://packages.cloud.google.com/apt google-compute-engine-bookworm-stable InRelease
Hit:3 https://deb.debian.org/debian bookworm-updates InRelease
Hit:4 https://deb.debian.org/debian bookworm-backports InRelease
Hit:6 https://deb.debian.org/debian-security bookworm-security InRelease
Hit:10 https://packages.cloud.google.com/apt cloud-sdk-bookworm InRelease
Hit:11 https://packages.cloud.google.com/apt google-cloud-ops-agent-bookworm-2 InRelease
Hit:7 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.29/deb  InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo apt install -y kubelet kubeadm kubectl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  conntrack cri-tools kubernetes-cni
Suggested packages:
  nftables
The following NEW packages will be installed:
  conntrack cri-tools kubeadm kubectl kubelet kubernetes-cni
0 upgraded, 6 newly installed, 0 to remove and 2 not upgraded.
Need to get 202 MB of archives.
After this operation, 704 MB of additional disk space will be used.
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 https://deb.debian.org/debian bookworm/main amd64 conntrack amd64 1:1.4.7-1+b2 [35.2 kB]
Get:3 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.29/deb  cri-tools 1.29.0-1.1 [20.1 MB]
Get:7 https://packages.cloud.google.com/apt cloud-sdk-bookworm/main amd64 kubectl amd64 1:561.0.0-0 [120 MB]
```

** We need to Lock few package versions: - else they get automatically updated **

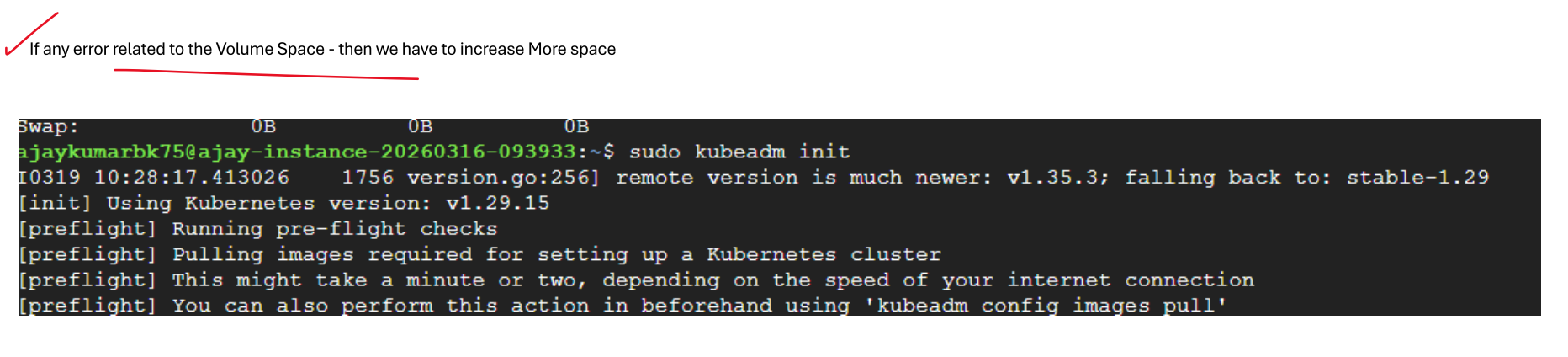


```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo apt-mark hold kubelet kubeadm kubectl
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

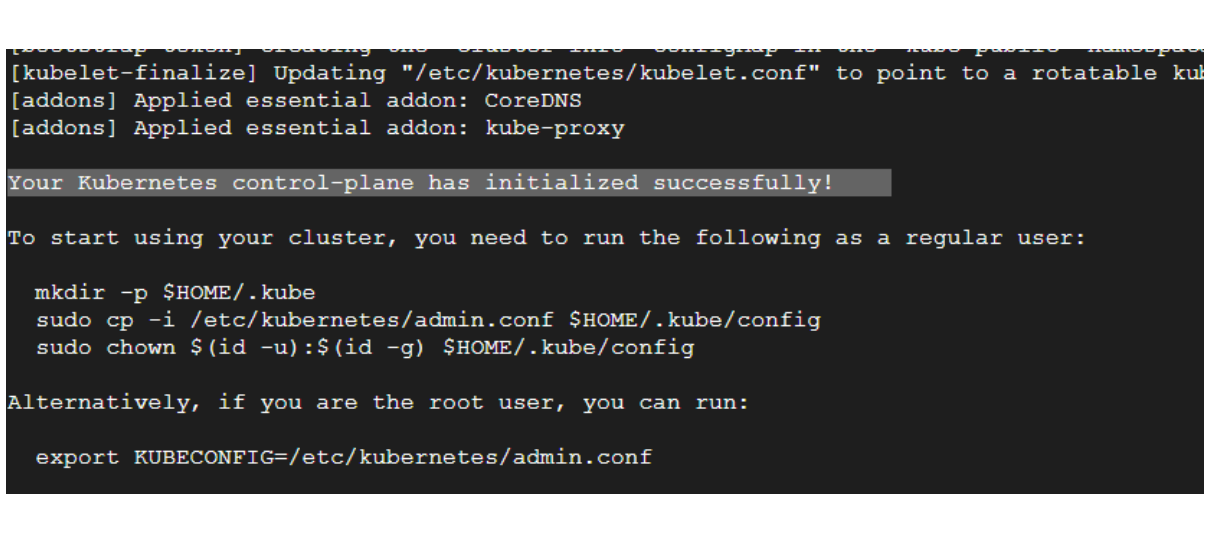
Step 8: Initialize Cluster

sudo kubeadm init

If any error related to the Volume Space - then we have to increase More space

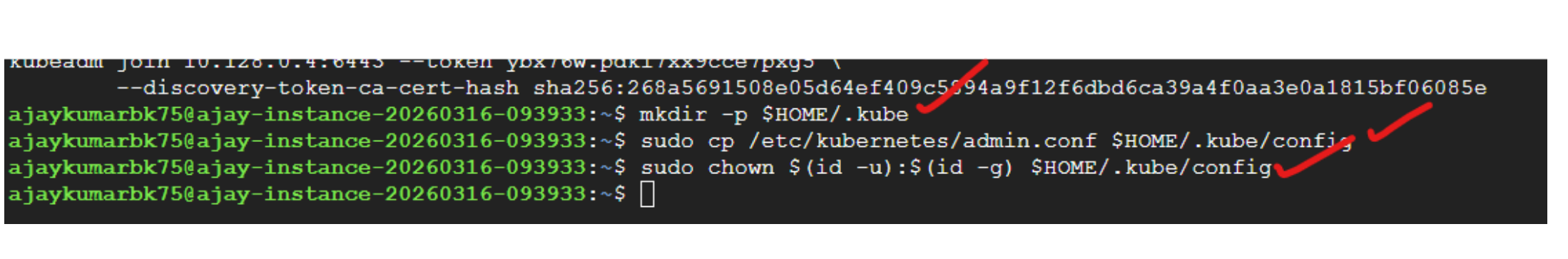


```
Swap:             0B          0B          0B
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo kubeadm init
I0319 10:28:17.413026    1756 version.go:256] remote version is much newer: v1.35.3; falling back to: stable-1.29
[init] Using Kubernetes version: v1.29.15
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
```



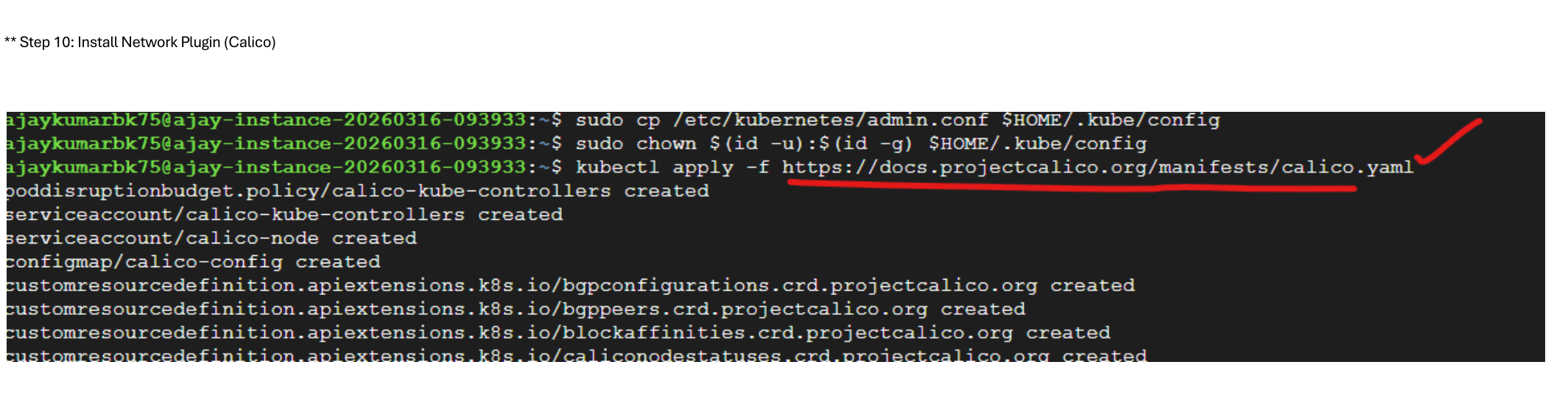
```
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable ku
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf
```

* Step 9: Configure kubectl -> Very important



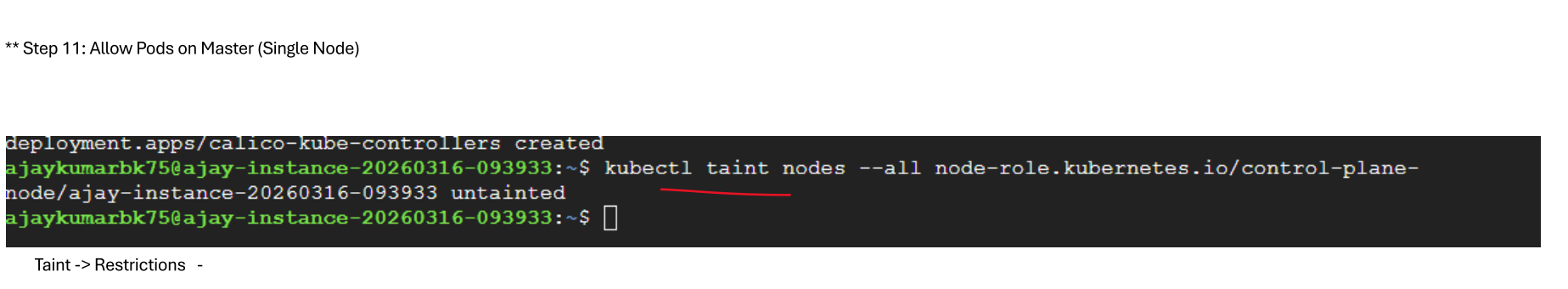
```
kubeadm join 10.128.0.4:6443 --token ybx76w.pdk17xx9cce7pxg5 \
        --discovery-token-ca-cert-hash sha256:268a5691508e05d64ef409c5594a9f12f6dbd6ca39a4f0aa3e0a1815bf06085e
ajaykumarbk75@ajay-instance-20260316-093933:~$ mkdir -p $HOME/.kube
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

** Step 10: Install Network Plugin (Calico)



```
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
ajaykumarbk75@ajay-instance-20260316-093933:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
serviceaccount/calico-node created
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
```

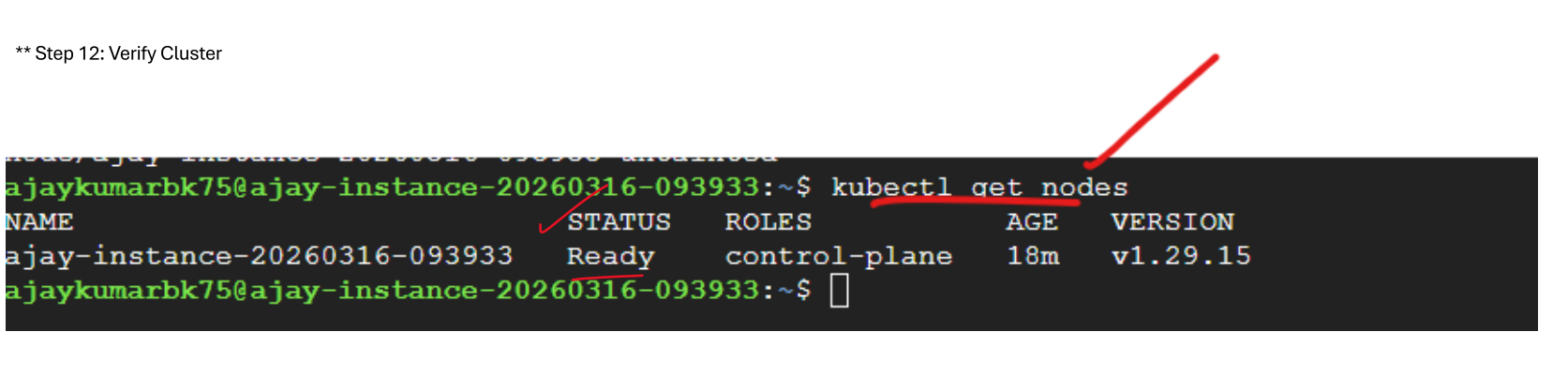
** Step 11: Allow Pods on Master (Single Node)



```
deployment.apps/calico-kube-controllers created
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl taint nodes --all node-role.kubernetes.io/control-plane-
node/ajay-instance-20260316-093933 untainted
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

Taint -> Restrictions -

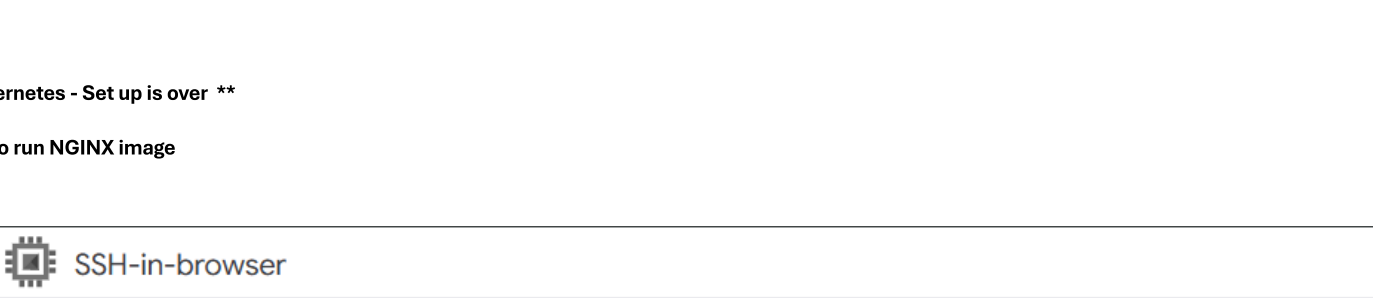
** Step 12: Verify Cluster



```
node/ajay-instance-20260316-093933 untainted
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl get nodes
NAME                            STATUS   ROLES           AGE   VERSION
ajay-instance-20260316-093933   Ready    control-plane   18m   v1.29.15
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

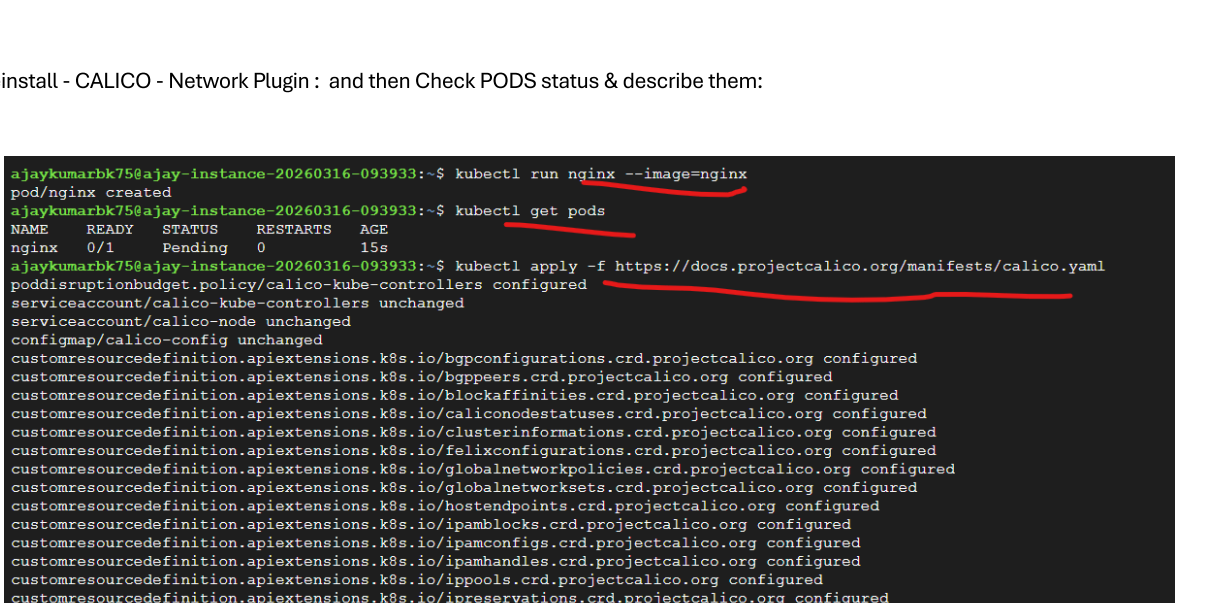
**** Kubernetes - Set up is over ****

**** Try to run NGINX image**



```
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl run nginx --image=nginx
pod/nginx created
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   0/1     Pending   0          15s
ajaykumarbk75@ajay-instance-20260316-093933:~$
```

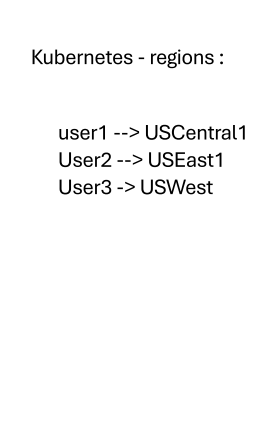
reinstall - CALICO - Network Plugin : and then Check PODS status & describe them:



```
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl run nginx --image=nginx
pod/nginx created
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   0/1     Pending   0          15s
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers configured
serviceaccount/calico-kube-controllers unchanged
serviceaccount/calico-node unchanged
configmap/calico-config unchanged
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/ipreservations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org configured
customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.org configured
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers unchanged
clusterrole.rbac.authorization.k8s.io/calico-node unchanged
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers unchanged
clusterrolebinding.rbac.authorization.k8s.io/calico-node unchanged
daemonset.apps/calico-node configured
deployment.apps/calico-kube-controllers unchanged
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl get pods -n kube-system
NAME                                                    READY   STATUS    RESTARTS   AGE
calico-kube-controllers-658d97c59c-h7h22                0/1     Pending   0          15m
calico-node-tgfj9                                       1/1     Running   0          15m
coredns-76f75df574-55whd                                0/1     Pending   0          29m
coredns-76f75df574-7tgkx                                0/1     Pending   0          29m
etcd-ajay-instance-20260316-093933                      1/1     Running   0          29m
kube-apiserver-ajay-instance-20260316-093933            1/1     Running   0          29m
kube-controller-manager-ajay-instance-20260316-093933   1/1     Running   0          29m
kube-proxy-5gvht                                        1/1     Running   0          29m
kube-scheduler-ajay-instance-20260316-093933            1/1     Running   0          29m
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   0/1     Pending   0          3m45s
ajaykumarbk75@ajay-instance-20260316-093933:~$ kubectl describe pods nginx
Name:             nginx
Namespace:        default
```

Kubernetes - regions :

user1 --> USCentral1
User2 --> USEast1
User3 -> USWest

GKE - Workflow : what we are doing :